SQL PLAYGROUND · SETUP GUIDE

Deploy on a fresh VPS

Ubuntu, Docker, nginx and ufw, behind Cloudflare: from a blank server to https://your-domain in about 45 minutes.

12
copy-and-paste steps

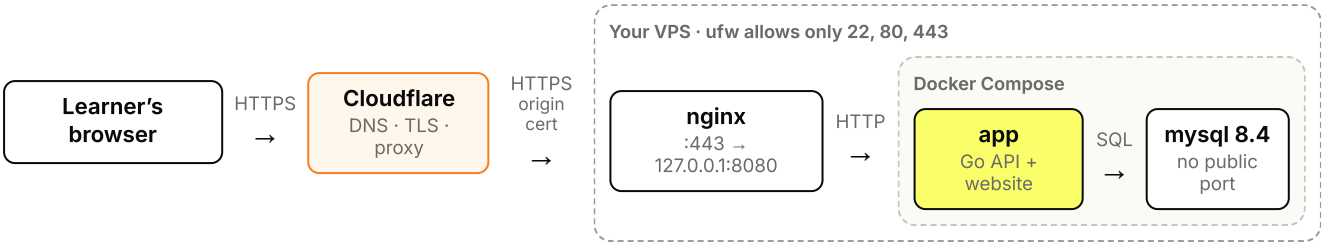
3
open ports: SSH, HTTP, HTTPS

0
database ports exposed

OVERVIEW

What you'll end up with

A fresh Ubuntu server running SQL Playground at **https://sql.example.com**, behind Cloudflare, with only SSH and HTTPS open to the internet. The database is never exposed, and every learner gets a private copy of the data inside MySQL.



Before you start

- [ ] A VPS with **Ubuntu 24.04 LTS** (22.04 also works), **8 GB RAM**, at least **20 GB free disk**, and SSH access as root (or a sudo user).
- [ ] Your domain on **Cloudflare**, with an A record pointing at the server. You'll check its settings in step 8.
- [ ] Access to the GitHub repository **darshan260802/sql-playground**.
- [ ] About **45 minutes**. Everything is copy and paste; run commands one block at a time.

Values used in this guide

In this guide	Replace with
`sql.example.com`	the hostname you'll use for the playground, e.g. *sql.yourdomain.com* (or just *yourdomain.com*)
`203.0.113.10`	your server's public IPv4 address
`darshan`	the Linux user you create in step 1 (any name works)

Placeholders are highlighted inside commands. Lines starting with `#` are comments; pasting them into the terminal is harmless.

1 Log in and create your admin user

Day-to-day work happens as a normal user with sudo, not as root.

From your own computer (Terminal on macOS/Linux, PowerShell on Windows), log in as root:

```
ssh root@203.0.113.10
```

Create the user, give it sudo, and copy root's SSH keys to it (if you log in with a key):

```
adduser darshan
usermod -aG sudo darshan
rsync --archive --chown=darshan:darshan ~/.ssh /home/darshan
```

Keep this root session open. In a **second** terminal on your computer, log in as the new user:

```
ssh darshan@203.0.113.10
```

> **TIP** If you log in with a password and have no SSH key yet, create one on your computer with `ssh-keygen -t ed25519` and copy it to the server with `ssh-copy-id darshan@203.0.113.10`. Step 11 turns off password logins, so set up a key before you get there.

From here on, run everything as **darshan**. Commands that need root start with `sudo`.

---

2 Update the system

Latest security fixes, automatic updates, and a small swap file as a safety net.

```
sudo apt update && sudo apt -y full-upgrade
sudo apt -y install git curl ca-certificates unattended-upgrades
sudo timedatectl set-timezone UTC
# choose "Yes" to install security updates automatically
sudo dpkg-reconfigure -plow unattended-upgrades
```

Swap (recommended): 2 GB of swap keeps the first build from running out of memory.

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

> **NOTE** If the upgrade asks whether to keep a modified configuration file (for example sshd_config), keep the version currently installed. If it installed a new kernel, reboot with `sudo reboot`, wait a minute, and log in again as darshan.

---

3 Turn on the firewall (ufw)

Block everything incoming except SSH. Web ports are opened in step 5.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw enable
sudo ufw status verbose
```

> **IMPORTANT** Always allow SSH **before** `ufw enable`, or you lock yourself out. If your SSH runs on a custom port, also run `sudo ufw allow 2222/tcp` (with your port).

> **NOTE** Some providers (AWS, Oracle, Hetzner Cloud, ...) also have a firewall in their web panel. If you use one, allow TCP 22, 80 and 443 there too.

4 Install Docker

The playground runs as two containers: the app and MySQL 8.4.

Add Docker's official package repository and install the engine and the compose plugin:

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
CODENAME=$(. /etc/os-release && echo "$VERSION_CODENAME")
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/ubuntu $CODENAME stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt -y install docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

Let your user run Docker without sudo, then log out and back in so it takes effect:

```
sudo usermod -aG docker darshan
exit
```

```
ssh darshan@203.0.113.10
docker run --rm hello-world
docker compose version
```

CHECK You should see "Hello from Docker!" and a Docker Compose version (v2.x).

NOTE Docker writes its own firewall rules, and ports it publishes skip ufw. This setup only publishes the app on `127.0.0.1` (localhost), and MySQL not at all, so nothing new reaches the internet. Keep it that way if you add other containers.

5 Install nginx

nginx receives HTTPS traffic from Cloudflare and hands it to the app.

```
sudo apt -y install nginx
sudo ufw allow 'Nginx Full'
# the default "Welcome to nginx" site isn't needed
sudo rm /etc/nginx/sites-enabled/default
sudo systemctl reload nginx
```

CHECK `sudo ufw status` now lists **OpenSSH** and **Nginx Full** (ports 80 and 443).

6 Get the code

Clone the repository into /opt/sql-playground.

```
sudo mkdir -p /opt/sql-playground
sudo chown darshan:darshan /opt/sql-playground
git clone --branch claude/zen-bohr-8sc57l \
  https://github.com/darshan260802/sql-playground.git /opt/sql-playground
```

> **NOTE** The code currently lives on the branch `claude/zen-bohr-8sc57l`. Once you merge it into `main`, clone with `--branch main` instead, and switch an existing server with `git checkout main && git pull`.

If the repository is private, the clone asks for credentials. Use a read-only deploy key instead:

```
ssh-keygen -t ed25519 -C "vps deploy key" -f ~/.ssh/github_deploy -N ""
cat ~/.ssh/github_deploy.pub
```

On GitHub open the repository → **Settings** → **Deploy keys** → **Add deploy key**, paste the key (leave "Allow write access" off), then:

```
printf 'Host github.com\n  IdentityFile ~/.ssh/github_deploy\n  IdentitiesOnly yes\n' \
  >> ~/.ssh/config
chmod 600 ~/.ssh/config
git clone --branch claude/zen-bohr-8sc57l \
  git@github.com:darshan260802/sql-playground.git /opt/sql-playground
```

7 Configure and start the playground

One settings file, one command. The first start builds the app and generates the dataset.

```
cd /opt/sql-playground/deploy
cp .env.example .env
chmod 600 .env
# a random password for MySQL's root account (copy the output)
openssl rand -hex 24
nano .env
```

In `.env`, set `MYSQL_ROOT_PASSWORD=` to the random value. Leave the rest as is for now (save with **Ctrl+O, Enter**, exit with **Ctrl+X**). The main settings:

Setting	Default	Meaning
`MYSQL_ROOT_PASSWORD`	none	required; the app uses it to create learner accounts
`APP_PORT`	8080	localhost port nginx talks to; change it if 8080 is taken
`COOKIE_SECURE`	true	keep true: the site is only used over HTTPS
`MAX_USERS`	20	sign-ups close at this many accounts
`USER_QUOTA_MB`	300	storage limit per learner database

Build and start (the first build takes 3–6 minutes):

```
docker compose up -d --build
docker compose logs -f app
```

Wait until you see `listening on :8080` (the dataset is generated first, about 30 seconds), then press **Ctrl+C** to stop following the logs. The containers keep running.

```
curl -s http://127.0.0.1:8080/api/health
docker compose ps
```

> **CHECK** The first command prints `{"status":"ok"}`, and both containers show **Up ... (healthy)**. They restart automatically after a reboot.

---

8 Configure Cloudflare

DNS through Cloudflare's proxy, end-to-end encryption, and a free 15-year origin certificate.

In the Cloudflare dashboard, open your domain. Menu names can move slightly over time; the settings stay the same.

8a · DNS record

DNS → Records: make sure there is an **A** record for your hostname (Name `sql` for sql.example.com, or `@` for the bare domain) pointing to `203.0.113.10`, with **Proxy status: Proxied** (orange cloud). If the server has IPv6, add an **AAAA** record the same way.

8b · Encryption mode

SSL/TLS → Overview: set the mode to **Full (strict)**.

8c · Edge certificates

SSL/TLS → Edge Certificates: turn on **Always Use HTTPS** and **Automatic HTTPS Rewrites**, and set **Minimum TLS Version** to **1.2**.

8d · Origin certificate (encrypts Cloudflare → your server)

SSL/TLS → Origin Server → Create Certificate. Keep "Generate private key and CSR with Cloudflare", key type **RSA (2048)**, list your hostname (e.g. `sql.example.com`; adding `*.yourdomain.com` also works), validity **15 years**, then **Create**. Leave the page open: it shows two text blocks.

```
sudo mkdir -p /etc/ssl/cloudflare
# paste the "Origin Certificate" block, save, exit
sudo nano /etc/ssl/cloudflare/sql-playground.pem
# paste the "Private Key" block, save, exit
sudo nano /etc/ssl/cloudflare/sql-playground.key
sudo chmod 600 /etc/ssl/cloudflare/sql-playground.key
```

> **IMPORTANT** Cloudflare shows the private key **only once**. Paste both blocks completely, including the BEGIN and END lines. The origin certificate is trusted by Cloudflare only, so keep the DNS record **Proxied**.

8e · Recommended extras

- **Speed → Optimization → Rocket Loader:** Off (it rewrites JavaScript and can break the app).
- **Caching → Configuration → Browser Cache TTL:** "Respect Existing Headers" (the app already sends the right cache headers).
- **Security → Bots → Bot Fight Mode:** if it's on and queries fail with "Can't reach the server", turn it off.

---

9 Connect nginx to the app

HTTPS with the origin certificate, plus real visitor IPs for the app's rate limits.

Install the site config from the repository and put your hostname in it:

```
# your hostname
DOMAIN=sql.example.com
sudo cp /opt/sql-playground/deploy/nginx/sql-playground-cloudflare.conf \
  /etc/nginx/sites-available/sql-playground.conf
sudo sed -i "s/sql\.example\.com/$DOMAIN/g" /etc/nginx/sites-available/sql-playground.conf
sudo ln -s /etc/nginx/sites-available/sql-playground.conf /etc/nginx/sites-enabled/
grep server_name /etc/nginx/sites-available/sql-playground.conf
```

CHECK The last command shows your hostname twice. If you changed `APP_PORT`, also edit the `server 127.0.0.1:8080;` line in that file.

Behind Cloudflare, every request appears to come from a Cloudflare server. This script makes nginx trust Cloudflare's published IP ranges, so the app sees each visitor's real IP. It also checks the nginx configuration and reloads it:

```
sudo /opt/sql-playground/deploy/scripts/cloudflare-realip.sh
```

CHECK The script ends with "nginx trusts NN Cloudflare ranges". If it stops with an error, see Troubleshooting at the end of this guide.

If `nginx -t` reports *"Address family not supported by protocol"*, the server has no IPv6. Remove the IPv6 listen lines and run the script again:

```
sudo sed -i '/listen \[::\]/d' /etc/nginx/sites-available/sql-playground.conf
sudo /opt/sql-playground/deploy/scripts/cloudflare-realip.sh
```

---

10 Test everything

Check the whole path, from the browser to the database.

1. Open **https://sql.example.com**. The playground loads with a padlock in the address bar.
2. Press **Run** on the welcome query. Results appear within a couple of seconds.
3. Click **Get started** and create your account. A yellow bar shows your private copy being prepared (about 10 seconds), then **My database** becomes active.
4. On **My database**, run `UPDATE products SET unit_price = unit_price * 1.1 WHERE category_id = 41;` and see "rows affected".
5. Open **Learn**, solve the first exercise, and press **Check**.

On the server:

```
curl -sI https://sql.example.com | grep -iE '^(HTTP|server)'
sudo tail -n 3 /var/log/nginx/access.log
```

CHECK You see `HTTP/2 200` and `server: cloudflare`. The access log starts with **your own** public IP, not a Cloudflare address (104.x, 172.64–71.x, …), which confirms step 9.

---

11 Harden the server

Recommended: lock the web ports to Cloudflare, turn off password logins, schedule backups.

11a · Only Cloudflare may reach ports 80 and 443

This stops anyone from bypassing Cloudflare by connecting to the server IP directly:

```
sudo /opt/sql-playground/deploy/scripts/cloudflare-realip.sh --ufw
sudo ufw status numbered
```

NOTE If you ever switch the DNS record to "DNS only" (grey cloud), open the ports again with `sudo ufw allow 'Nginx Full'`.

11b · SSH: keys only, no root login

Do this only after logging in as darshan **with a key** works:

```
sudo tee /etc/ssh/sshd_config.d/99-hardening.conf > /dev/null <<'EOF'
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
EOF
sudo sshd -t && sudo systemctl restart ssh
```

IMPORTANT Keep your current session open and test a **new** SSH login before closing it. If the new login fails, undo the change by deleting that file from the open session.

11c · Backups and Cloudflare range updates on a schedule

```
sudo apt -y install fail2ban
sudo crontab -e
```

Add these two lines at the bottom (a daily backup at 03:17 and a monthly refresh of Cloudflare's ranges), then save:

```
17 3 * * * /opt/sql-playground/deploy/scripts/backup.sh >/dev/null 2>&1
23 4 1 * * /opt/sql-playground/deploy/scripts/cloudflare-realip.sh >/dev/null 2>&1
```

Backups land in `/var/backups/sql-playground/` (the last 14 are kept). They contain accounts, saved queries, history and lesson progress. Learners' database copies aren't included; the app rebuilds them automatically after a restore. Copy a backup off the server now and then, e.g. from your computer:

```
scp darshan@203.0.113.10:/var/backups/sql-playground/app-*.sql.gz .
```

12 Everyday operations

Run these from /opt/sql-playground/deploy.

Task	Command
Status / logs	`docker compose ps` `docker compose logs -f app`
List accounts and storage	`docker compose exec app /app/sqlpg admin list-users`
Reset a forgotten password (prints a new one)	`docker compose exec app /app/sqlpg admin reset-password EMAIL`
Restore a learner's data to the original	`docker compose exec app /app/sqlpg admin reset-db EMAIL`
Delete an account and its database	`docker compose exec app /app/sqlpg admin delete-user EMAIL`
Update to the latest code	`git pull && docker compose up -d --build` `docker image prune -f`
Change a setting in .env	`nano .env && docker compose up -d`
Restart / stop	`docker compose restart app` `docker compose down`
Disk usage	`df -h /` `docker system df`
Back up now	`sudo ./scripts/backup.sh`

IMPORTANT Never run `docker compose down -v`: the `-v` deletes the MySQL volume, which holds every account and database.

Restore a backup (same server or a new one)

Set up the new server with steps 1–9 first, then:

```
cd /opt/sql-playground/deploy
gunzip -c app-2026-01-31.sql.gz | docker compose exec -T mysql \
  sh -c 'MYSQL_PWD=$MYSQL_ROOT_PASSWORD mysql -uroot'
docker compose restart app
```

Accounts, saved queries and progress come back, and each learner's database is rebuilt in the background (about 10 seconds each).

What to expect

MySQL uses about 1.2 GB of RAM and the app about 30 MB. Disk: about 1.1 GB base, plus 148 MB per learner (up to the 300 MB limit), so 20 learners need roughly 3–6 GB.

? Troubleshooting

Cloudflare error pages show a number. Start there.

Symptom	Likely cause	Fix
Cloudflare **521** "Web server is down"	nginx stopped, or a firewall blocks 443	`sudo systemctl status nginx`; `sudo ufw status` must allow 443 (Nginx Full or Cloudflare rules); check the provider's firewall
Cloudflare **522** "Connection timed out"	wrong IP in the DNS record, or a provider firewall drops traffic	compare the A record with the server's IP; open 80/443 in the provider panel
Cloudflare **525** "SSL handshake failed"	nginx isn't serving HTTPS on 443	`sudo nginx -t`; check the site is in sites-enabled and the certificate paths exist
Cloudflare **526** "Invalid SSL certificate"	Full (strict) but the certificate isn't the origin certificate, or it doesn't list this hostname	create a new origin certificate that includes the hostname (step 8d) and replace both files
"Too many redirects"	SSL/TLS mode is "Flexible"	set **Full (strict)** (step 8b)
nginx **502 Bad Gateway**	the app isn't running, or the port doesn't match	`docker compose ps`; `docker compose logs app`; APP_PORT must match the upstream port in the nginx file
`nginx -t`: "Address family not supported by protocol"	the server has no IPv6	remove the `listen [::]` lines (step 9 note)
Signed out right after signing in	the site was opened over http://	always use https:// (turn on Always Use HTTPS, step 8c); keep COOKIE_SECURE=true
Everyone gets "You're running queries very quickly"	real visitor IPs aren't set up, so all traffic looks like one Cloudflare IP	run `sudo /opt/sql-playground/deploy/scripts/cloudflare-realip.sh`
Queries fail with "Can't reach the server"	Bot Fight Mode or a WAF rule challenges API calls	turn off Bot Fight Mode, or skip security for `/api/` paths
"This playground is full"	MAX_USERS reached	raise MAX_USERS in .env and run `docker compose up -d`, or delete unused accounts
Build stops with "Killed"	not enough memory while building	add swap (step 2) and run the build again
MySQL container keeps restarting	memory limit or a broken .env	`docker compose logs mysql`; check MYSQL_ROOT_PASSWORD; lower `innodb_buffer_pool_size` in deploy/mysql/playground.cnf

A Files at a glance

Where everything lives on the server.

Path	What it is
`/opt/sql-playground`	the repository (code, Dockerfile, deploy files)
`/opt/sql-playground/deploy/.env`	your settings and the MySQL root password (keep private)
`/opt/sql-playground/deploy/docker-compose.yml`	the two containers and their limits
`/opt/sql-playground/deploy/mysql/playground.cnf`	MySQL tuning (buffer pool, safety settings)
`/etc/nginx/sites-available/sql-playground.conf`	the nginx site (HTTPS + proxy to the app)
`/etc/nginx/conf.d/cloudflare-realip.conf`	generated list of trusted Cloudflare ranges
`/etc/ssl/cloudflare/`	the Cloudflare origin certificate and private key
`/var/backups/sql-playground/`	daily backups of accounts and progress
Docker volume `sql-playground_mysql-data`	all MySQL data (core dataset, learner copies, app tables)

The order, in one glance

1. user + SSH key
2. updates + swap
3. ufw: SSH
4. Docker
5. nginx + ufw: web
6. clone
7. .env + compose up
8. Cloudflare: DNS, Full (strict), origin cert
9. nginx site + real IPs
10. test
11. harden + backups